

SkillSync - Sprint 3 Testing Deliverable

1) Test Plan (Sprint 3)

1.1 Scope

Testing in this sprint focuses on authentication quality and reliability:

- Password hashing and verification logic (PasswordHasher)
- Signup and login decision logic (AuthService)

UI/controller and database end-to-end checks are listed as manual black-box tests.

1.2 Test Objectives

- Confirm valid users can sign up and log in.
- Ensure invalid credentials and malformed inputs are rejected safely.
- Verify duplicate account handling and database-failure handling paths.
- Validate password security behavior (hashing + verification).

1.3 Test Strategy

- **Black-box testing:** equivalence partitioning, boundary-focused inputs, and error guessing on auth features.
- **White-box testing:** statement and branch-focused tests on PasswordHasher and AuthService.
- **Execution approach:** run automated Java test classes with pass/fail assertions from command line.

1.4 Entry and Exit Criteria

- **Entry:** project compiles; auth source files available.
- **Exit:** all planned automated tests pass; major auth paths exercised; findings recorded.

1.5 Environment

- Java runtime and compiler available locally.
- Source path: src/
- Command used:
 - javac PasswordHasher.java User.java DatabaseConfig.java DatabaseManager.java UserRepository.java AuthService.java PasswordHasherTest.java AuthServiceTest.java
 - java PasswordHasherTest
 - java AuthServiceTest

2) Black-Box Test Cases

2.1 Equivalence Partitioning

- **BB-01 Valid login partition**

Input: existing identifier + correct password

Expected: login success (true)

- **BB-02 Invalid password partition**

Input: existing identifier + wrong password

Expected: login denied (false)

- **BB-03 Unknown user partition**

Input: non-existing identifier + any password

Expected: login denied (false)

- **BB-04 Duplicate signup partition**

Input: username/email already registered

Expected: signup failure with duplicate message

- **BB-05 New user signup partition**

Input: unique username/email/password/role

Expected: signup success

2.2 Boundary-Oriented Cases

- **BB-06 Null password at verification boundary**

Input: plainPassword = null

Expected: verify(...) returns false

- **BB-07 Null stored hash boundary**

Input: storedHash = null

Expected: verify(...) returns false

- **BB-08 Malformed stored hash boundary**

Input: non-PBKDF2 format string

Expected: verify(...) returns false

2.3 Error Guessing

- **BB-09 Database duplicate-key SQL error**

Simulate SQL error codes 2627/2601 during signup

Expected: user-friendly duplicate message

- **BB-10 Generic SQL exception during login**

Simulate DB failure while loading hash

Expected: login fails safely (false), no crash

3) White-Box Testing Evidence

3.1 Modules and Coverage Criteria

- PasswordHasher: statement/branch-focused checks across happy path and error-handling branches.
- AuthService: statement/branch-focused checks for success and all major failure branches.

3.2 Automated White-Box Tests Implemented

- src/PasswordHasherTest.java
 - Verifies hash prefix/format behavior.
 - Verifies correct password succeeds.
 - Verifies wrong password, nulls, malformed hash fail.
- src/AuthServiceTest.java
 - Verifies signup success path.
 - Verifies duplicate SQL error path.
 - Verifies generic DB error path.
 - Verifies login success, wrong password, unknown user, and DB exception paths.

3.3 Testability Improvement Made

A testability issue was identified: AuthService created UserRepository internally, which blocked deterministic unit testing.

Fix applied: constructor-based dependency injection added in src/AuthService.java while keeping default behavior unchanged for production code.

4) Defect/Bug Report Summary

4.1 Defects Identified

- **DEF-01 (Testability defect):** AuthService tightly coupled to concrete repository instance.
- Impact: difficult to test exception and branch behavior.
- Status: **Fixed** in Sprint 3 by constructor injection.

4.2 Functional Defects During This Test Cycle

- No new functional defects found in tested authentication logic during automated execution.

5) Test Execution Results and Observations

5.1 Execution Evidence

Automated run output:

- PasswordHasherTest passed.
- AuthServiceTest passed.

5.2 Observations

- Password handling is secure-by-design in tested paths (hash + verify, no plaintext comparison).
- Auth flow degrades safely on database exceptions (returns failure instead of crashing).
- Duplicate-account conditions return clear user-facing feedback.

6) How Testing Improved System Quality

- Added repeatable automated checks for key authentication paths.
- Increased confidence in both positive and negative auth scenarios.
- Improved maintainability through dependency injection in AuthService.
- Reduced regression risk for future Sprint 3+ changes in login/signup logic.